



ObjectPoolManager

- Один из базовых и чаще всего используемых паттернов программирования в сфере разработки игр. – ObjectPool

ObjectPool.

Object pool – это очередь / стек / лист: (список) объектов, которые создаются в определенный момент, скрываются и лежат в списке весь жизненный цикл проекта.

На создание (аллокацию) каждого объекта требуются ресурсы. Ресурсы требуются и на очистку уничтоженных объектов, которые становятся жертвой сборщика мусора. Если таких объектов категорически много и/или они появляются \ уничтожаются слишком быстро, то игра может начать фризить (тормозить). Чтобы избежать этого нужно избавиться от создания и начать переиспользовать объекты, просто сбрасывая и/или обновляя их характеристики, тем самым кратно сократив затраты и критически снизив шансы на “томрожение” игры.

```
1 using UnityEngine;
2 using System.Collections.Generic;
3 using UnityEngine.SceneManagement;
4 public class ObjectPool : MonoBehaviour
5 {
6     private GameObject prefab;
7     private Transform parent;
8     private int expandSize;
9
10    private Queue<GameObject> pool = new Queue<GameObject>();
11
12    public ObjectPool(GameObject prefab, int initialSize, int expandSize)
13    {
14        this.prefab = prefab;
15        this.parent = parent;
16        this.expandSize = Mathf.Max(5, expandSize);
17
18        CreateBatch(initialSize);
19    }
20
21    /*
22     * Класс ObjectPool у меня будет работать через очередь, которая будет хранить
23     * список объектов. Через конструктор я могу задать стартовое число объектов
24     * а также передаю экземпляр объекта, число для последующих расширений списка
25     * (если переиспользуемые объекты закончатся, то класс разом создаст ещё
26     * таких же и очередь уже будет больше).
27
28     * Также в конструктор нужно передать объект, который будет содержать в себе
29     * все экземпляры (папка в иерархии Unity).
30     */
31
32    private void CreateBatch(int count)
33    {
34        for (int i = 0; i < count; i++)
35        {
```

```

36         GameObject obj = GameObject.Instantiate(prefab, parent);
37         obj.SetActive(false);
38         pool.Enqueue(obj);
39     }
40 }
41
42 // В CreateBatch() я передаю в Instantiate экземпляр и родителя (папку)
43 // также сразу отключаю объект, чтобы он не обрабатывался в фоне, и доб
44 // очередь.
45
46 public GameObject Get()
47 {
48     if (pool.Count == 0)
49     {
50         CreateBatch(expandSize);
51     }
52
53     GameObject obj = pool.Dequeue();
54     obj.transform.SetParent(null);
55     SceneManager.MoveGameObjectToScene(obj, SceneManager.GetActiveScene());
56     obj.SetActive(true);
57
58     return obj;
59 }
60
61 /*
62  * Метод Get() проверять есть ли объект в списке, в случае отсутствия
63  * - расширяет список. Если объекты есть → достаёт из списка, делает актив
64  * достаёт из родителя (папку). В моём случае сцена с уровнем подгружается
65  * - поверх persistent-boot, а пулы объектов хранятся там.
66  * Чтобы правильно учесть некоторую логику (правильное расположение на кар
67  * вычисление зон видимости башен и прочее) - я перемещаю объект на текущу
68  * активную сцену.
69  */
70
71 public void Return(GameObject obj)
72 {
73     obj.SetActive(false);
74     SceneManager.MoveGameObjectToScene(obj, gameObject.scene);
75     obj.transform.SetParent(parent);
76     pool.Enqueue(obj);
77 }
78
79 // Return() - возвращает объект на прежнюю сцену, в родителя,
80 // выключает его и кладёт обратно в список.
81
82 }
83

```

PoolManager.

Нужен кто-то, кто будет создавать и контролировать пулы, а также через себя позволять абстрактно обращаться к ним и брать объекты.

Во-первых, PoolConfig - будет представлять из себя экземпляр одного Pool (префаб, стартовое количество, количество под расширение).

```

1 using UnityEngine;
2
3 [CreateAssetMenu(fileName = "PoolConfig", menuName = "Pooling/Pool Config")]
4 public class PoolConfig : ScriptableObject
5 {
6     public GameObject prefab;
7
8 }
9

```

```

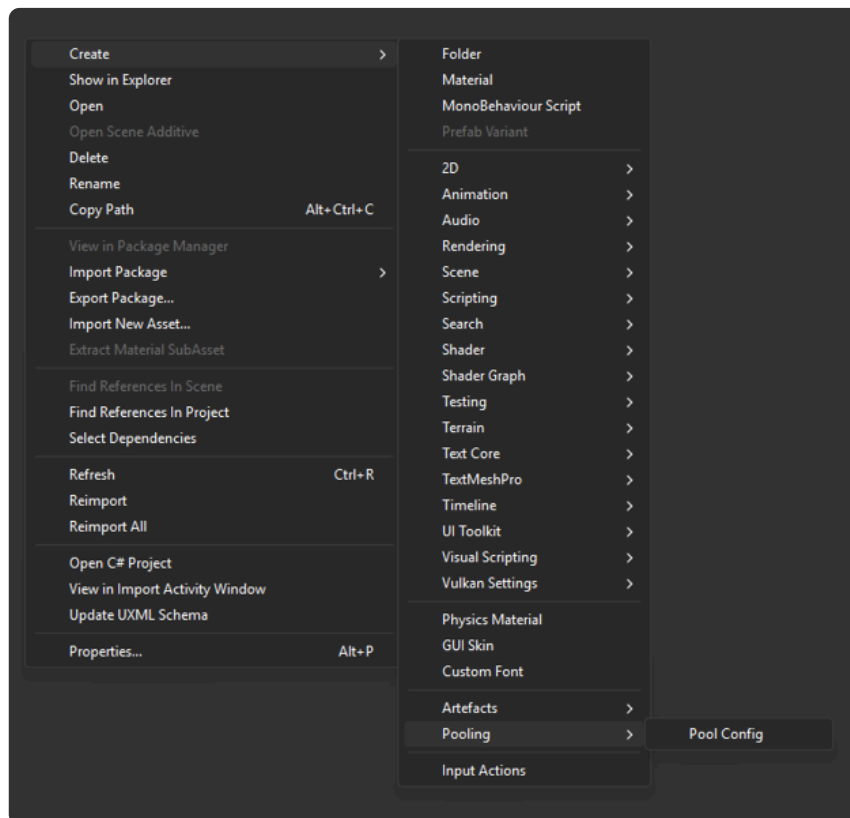
7 public int initialSize = 10;
8 public int expandSize = 5;
9 }

```

Я делаю ScriptableObject, чтобы создавать их отдельно и хранить как шаблоны под пулы.



Inspector.



Create Pool Config.

Далее мы перебираем все конфиги и проверяем наличие у префаба IPoolIdentity:

```

1 public interface IPoolIdentity
2 {
3     string GetPoolId();
4 }

```

```

1 public class EnemyController : MonoBehaviour, IPoolIdentity
2 {
3     ...
4     public string GetPoolId() => Type.ToString();
5 }

```

```

1 using System.Collections.Generic;
2 using UnityEngine;
3
4 public class PoolManager : Manager<PoolManager>
5 {
6     [SerializeField] public PoolConfig[] poolConfigs;
7
8     private Dictionary<string, ObjectPool> pools = new Dictionary<string, ObjectPool>();
9
10    // Словарь будет хранить название Pool, а также сам класс ObjectPool,
11    // который в свою очередь и будет хранить объекты.
12
13
14    private Transform poolsRoot;
15
16    private void Start()
17    {
18
19        poolsRoot = new GameObject("Pools").transform;
20        poolsRoot.SetParent(transform);
21
22        foreach (var config in poolConfigs)
23        {
24            if (!config.prefab.TryGetComponent<IPoolIdentity>(out var identity))
25            {
26                Debug.LogError($"Prefab {config.prefab.name} has no IPoolIdentity component");
27                continue;
28            }
29
30            string poolId = identity.GetPoolId();
31
32            if (pools.ContainsKey(poolId))
33            {
34                Debug.LogError($"Duplicate poolId: {poolId}");
35                continue;
36            }
37
38            var parent = new GameObject($"{poolId}_Pool").transform;
39            parent.SetParent(poolsRoot);
40
41            var pool = new ObjectPool(
42                config.prefab,
43                config.initialSize,
44                config.expandSize,
45                parent
46            );
47
48            pools.Add(poolId, pool);
49        }
50    }
51
52    // В старте мы создаём объект папку, внутрь которого положим сам менеджер
53    // а также в будущем будем сюда складывать другие пулы.
54    // В случае с врагами, Id для пула = их Type (EnemyType).
55
56    /*
57     * Мы проверяем, есть ли в словаре ключ Id. Если нет → создаём пустой объект
58     * с названием {Id}_Pool. И кладем его в общую папку пулов.
59     * Создаём пул, передав ему из конфига префаб, стартовое количество и расшаренный
60     * а также только что созданную пустышку - папку, под которым пул и будет

```

```

61  объекты в иерархии.
62  */
63
64  public GameObject Get(string poolId)
65  {
66      if (!pools.TryGetValue(poolId, out var pool))
67      {
68          Debug.LogError($"Pool with id '{poolId}' not found");
69          return null;
70      }
71      return pool.Get();
72  }
73
74  public void Return(GameObject obj)
75  {
76      if (!obj.TryGetComponent<IPoolIdentity>(out var identity))
77      {
78          Debug.LogError($"Object {obj.name} has no IPoolIdentity");
79          return;
80      }
81
82      string poolId = identity.GetPoolId();
83
84      if (!pools.TryGetValue(poolId, out var pool))
85      {
86          Debug.LogError($"Pool with id '{poolId}' not found for obje
87          return;
88      }
89
90      pool.Return(obj);
91  }
92  }

```

При старте игры мы будем хранить все необходимые списки объектов, которые сможем переиспользовать.

Для получения мы просто обращаемся к менеджеру пулов через вызов Get(айди объекта). Тот в свою очередь обращается к словарию и ищет необходимый пул. Если текущий есть - возвращает объект.

Аналогично с возвращением Return(GameObject). Чтобы не отправить в список какой-то бродячий объект - мы проведём две проверки: проверим наличие Id, а также проверим есть ли такой айди в словаре.

Можно ещё было реализовать интерфейс с методами, которые бы вызывались при Get(), Return(). Однако я решил ограничиться OnEnable() - при включении объекта, которое происходит в любом случае, т.к. Get вызывает SetActive(true).

В OnEnable() я вызываю Reset(), который в свою очередь и сбрасывает характеристики объекта (к примеру - очки здоровья врага).

Таким образом я реализовал базовый и почти всегда нужный паттерн ObjectPool. Объекты переиспользуются, постоянных аллокаций (выделений памяти), а также бесконечной сборки мусора не происходит.